ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Hans Jonathan**
- **Chinese Name: 熊明漢**
- **Email: s115065422@m115.nthu.edu.tw**
- **Student ID: 115065422**
- **University: NTHU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1.1 How many documents are in this specific category?
1.2 From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1.1 594 documents

1.2 Filtering lets us focus on patterns within one topic. If we mix medicine, religion, and etc, common word combinations may come from unrelated topics. Looking at one category makes the patterns easier to interpret and more accurate.

AI used for this answer: No

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question 2.1: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

2.1 Dropping those rows would throw away 40% of the documents even though their text is still useful. It could also bias the dataset if some categories are more likely to have missing labels. One alternative is semi-supervised learning, which uses both labeled and unlabeled documents. Any predicted labels should still be treated as estimates, not known answers.

AI used for this answer: Yes

If yes, how: AI explained the concepts and drafted this answer.

External references (links): scikit-learn semi-supervised learning

---

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

3.1 Do the exact category proportions perfectly match the original?
3.2 What is the specific risk of simple random sampling on a highly imbalanced dataset?
Answer:

3.1 No, the proportions are close but do not match exactly. In this run, the original versus sampled percentages were: soc.religion.christian 26.54% vs 24.80%; sci.med 26.32% vs 26.00%; comp.graphics 25.88% vs 27.80%; and alt.atheism 21.27% vs 21.40%.

3.2 A small minority category might end up with very few documents, or none at all. This makes it harder to learn patterns for that category. Stratified sampling helps keep the category proportions close to the original dataset.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results. AI also helped edit the experiment code.

External references (links): scikit-learn stratified sampling

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

4.1 Does the total number of features (columns) increase or decrease?
4.2 Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?
Answer:

4.1 The number of features increases from 35,482 with stop_words='english' to 35,788 with stop_words=None. That adds 306 features. Both matrices still have 2,257 documents.

4.2 Sparsity decreases from 99.6772% to 99.5470%. Adding common words creates more nonzero entries because those words appear in many documents. The matrix is still very sparse overall.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results. AI also helped edit the experiment code.

External references (links): scikit-learn CountVectorizer

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

5.1 What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?

5.2 In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?

5.3 Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35,482
0.01	5,089
0.05	1,491

5.1 The vocabulary gets smaller as the threshold increases. It goes from 35,482 terms at 0.0 to 5,089 at 0.01, then 1,491 at 0.05.

5.2 Very rare words with small counts usually have low variance because their columns are almost all zeros. Words with nearly the same count in every document also have low variance. A word being common does not automatically mean its variance is low.

5.3 Features that barely change usually give little information for telling documents apart, so removing them can reduce memory use and computation. It is not guaranteed to preserve accuracy, though. A rare word could still be useful for a small category, so model performance should be checked.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results. AI also helped edit the experiment code.

External references (links): scikit-learn variance filtering

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

6.1 Are the top-correlated words generic stop words or domain-specific terminology for graphics?
6.2 What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
6.3 In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

6.1 The results are mixed, but Spearman gives more graphics-related terms. The Pearson top 10 are thanks, keywords, looking, vga, video, card, hi, host, algorithm, and nntp. The Spearman top 10 are graphics, files, image, 3d, program, thanks, file, windows, ftp, and format. Words like graphics, image, and 3d fit the topic. Thanks and hi are general words, while host and nntp can come from message headers. These are not all standard stop words.

6.2 Pearson measures a linear relationship using the actual values. Spearman applies correlation to the ranks, so it measures whether the values tend to increase or decrease together, even when the relationship is not linear.

6.3 Suppose a word appears in most graphics documents and rarely in other categories, but a few graphics documents repeat it hundreds of times. The ranks can still separate the categories well, giving a high Spearman correlation. Those extreme counts can weaken the linear relationship with the binary category label, giving a lower Pearson correlation.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results. AI also helped edit the experiment code.

External references (links): SciPy Pearson correlation; SciPy Spearman correlation

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

7.1 How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?

7.2 How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?

7.3 Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

7.1 For alt.atheism, lowering minSup from 9 to 3 increased the runtime from 0.049866 seconds to 1.417059 seconds, and the number of maximal patterns from 710 to 9,547. The lower threshold allows less common patterns, so the algorithm has more possibilities to explore. These are the results of this run; runtime can vary between runs.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

7.2 k controls how many of the highest-support patterns Top-K FAE aims to return. minSup sets the minimum number of transactions containing a pattern; the data then determines how many patterns qualify. Here, k=100 returned 100 patterns in 0.018150 seconds, while k=500 returned 500 patterns in 0.101609 seconds.

Answer (3) — Maximal vs standard frequent patterns:

7.3 A maximal frequent pattern has no larger frequent pattern containing it. For example, if {a, b, c} is frequent and maximal, we do not need to list all its frequent subsets separately. This makes the output shorter, but it does not preserve the exact support of every subset.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results. AI also helped edit the experiment code.

External references (links): PAMI frequent pattern mining

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

8.1 If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
8.2 What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

8.1 X_counts has 35,482 features, so each document is a point with 35,482 coordinates. A normal scatter plot only has two or three axes. Choosing just two word columns would leave out almost all the other information, so it would not show the full document relationships.

8.2 We first use dimensionality reduction, such as PCA, t-SNE, or UMAP, to turn each document into two or three coordinates that we can plot.

AI used for this answer: Yes

If yes, how: AI explained the concepts and drafted this answer.

External references (links): scikit-learn dimensionality reduction

---

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

9.1 Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
9.2 Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
9.3 Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

9.1 In the saved 2D plots, PCA has heavy overlap near the center and a few faraway points. t-SNE and UMAP spread the documents out more and show some smaller islands or clearer regions. However, the four categories still overlap; the plots do not show four

completely separate category clusters. UMAP gives some clearer category regions, but the separation is only partial.

9.2 PCA keeps directions with the most overall variance, which are not necessarily the directions that separate the categories. With raw word counts, document length and very large counts can dominate. PCA is also linear. t-SNE and UMAP can capture nonlinear neighborhood structure, although they still do not guarantee clean category separation.

9.3 t-SNE mainly tries to preserve nearby relationships. The distance between separate islands does not reliably measure how different their topics are. The gaps can also change with settings or initialization, so a cluster being twice as far away does not mean it is twice as different.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results.

External references (links): scikit-learn PCA; scikit-learn t-SNE

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

10.1 How many binary columns are created to represent the 4 categories?
10.2 If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
10.3 How does one-hot binarization eliminate this false assumption?

Answer:

10.1 Four binary columns are created, one for each category. The saved output confirms an encoded matrix shape of (2257, 4).

10.2 If the category codes are treated as numerical input features, the algorithm can assume a meaningful order and distance. For example, it would treat category 3 as closer to category 2 than to category 0. These topic labels do not have that kind of numerical relationship.

10.3 One-hot binarization gives each category its own column. A document gets 1 in its category column and 0 in the others. This removes the artificial order and unequal distances created by the integer codes.

AI used for this answer: Yes

If yes, how: AI drafted this answer and interpreted the saved notebook results. AI also helped edit the experiment code.

External references (links): scikit-learn LabelBinarizer
